Uczenie głębokie – Laboratorium 1

Igor Józefowicz 193257 Kamil Śliwiński 193740

6 listopada 2026

1 zadanie 1

```
## Wynik dla jądra k1:
[[-4.,  -6.,   4.],

 [-20., -14.,  20.],

 [-20., -10.,  20.]]
 ## Wynik dla jądra k2:
[[ 9.,  6., 16.],

 [21., 27., 30.],

 [14., 30., 27.]]
 ## Wynik dla jądra k3:
 [[  8.,  15.,   9.],

 [  7.,   8.,   9.],

 [ -8., -15.,  -9.]]

```

2 zadanie 2

```
import matplotlib.pyplot as plt
import os

kernels['k4'] =   np.array([
                            [1., 0., 0.],
                            [0., 1., 0.],
                            [0., 0., 1.],
                            ])


image_path = 'lab1_1.jpg'
```

```
if os.path.exists(image_path):
    img_input = read_image(image_path, 200, 200)
else:
    print(f"Plik {image_path} nie został znaleziony! Używam losowego obrazu testowego.")
    img_input = np.random.rand(200, 200, 1) * 255

# Obliczenie wyników splotów
out_k1 = apply_conv2d(img_input, kernels['k1'])
out_k2 = apply_conv2d(img_input, kernels['k2'])
out_k3 = apply_conv2d(img_input, kernels['k3'])
out_k4 = apply_conv2d(img_input, kernels['k4'])

# Wyświetlenie obrazów
fig, axs = plt.subplots(1, 5, figsize=(15, 5))

# Oryginalny obraz
axs[0].imshow(img_input.squeeze(), cmap='gray')
axs[0].set_title('Oryginalny obraz')
axs[0].axis('off')

# Wynik dla k1
axs[1].imshow(out_k1[0, :, :, 0], cmap='gray')
axs[1].set_title('Filtr k1 (pionowy)')
axs[1].axis('off')

# Wynik dla k2
axs[2].imshow(out_k2[0, :, :, 0], cmap='gray')
axs[2].set_title('Filtr k2 (wyostrzenie)')
axs[2].axis('off')

# Wynik dla k3
axs[3].imshow(out_k3[0, :, :, 0], cmap='gray')
axs[3].set_title('Filtr k3 (poziomy)')
axs[3].axis('off')

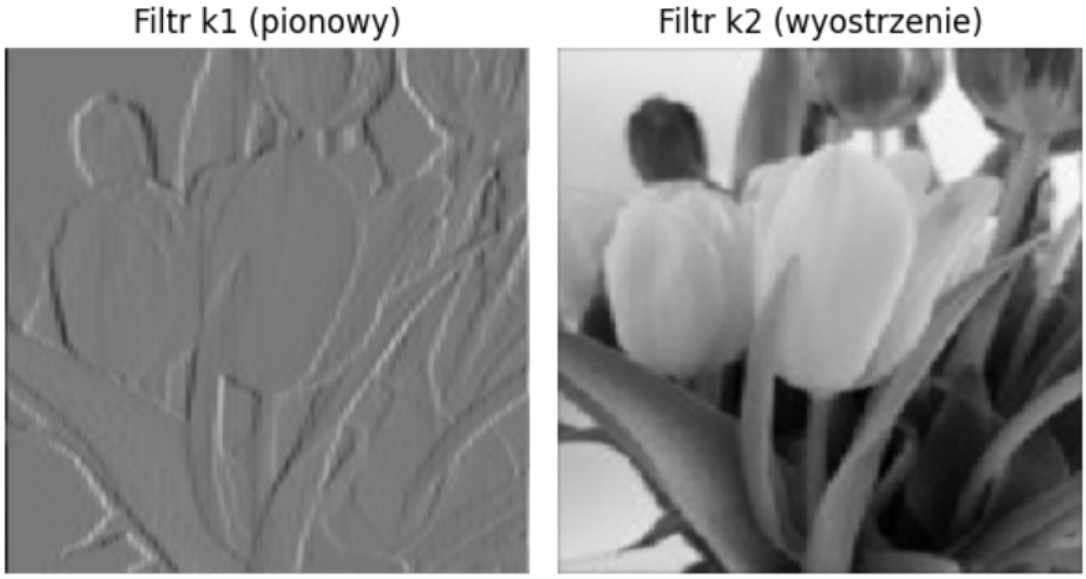
# Wynik dla k4
axs[4].imshow(out_k4[0, :, :, 0], cmap='gray')
axs[4].set_title('Filtr k4')
axs[4].axis('off')

plt.tight_layout()
plt.show()

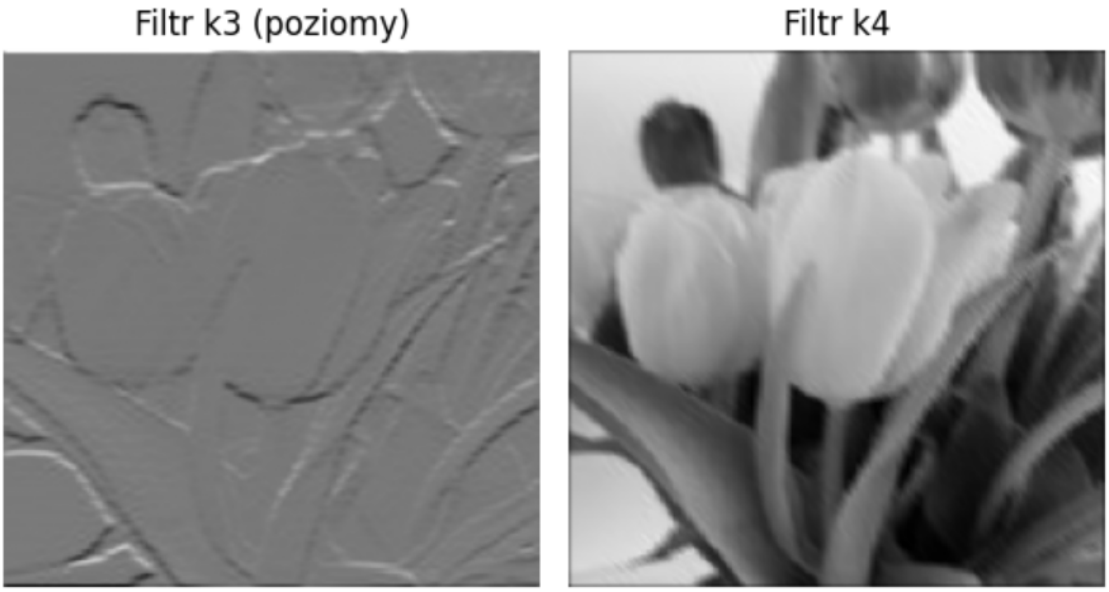
```



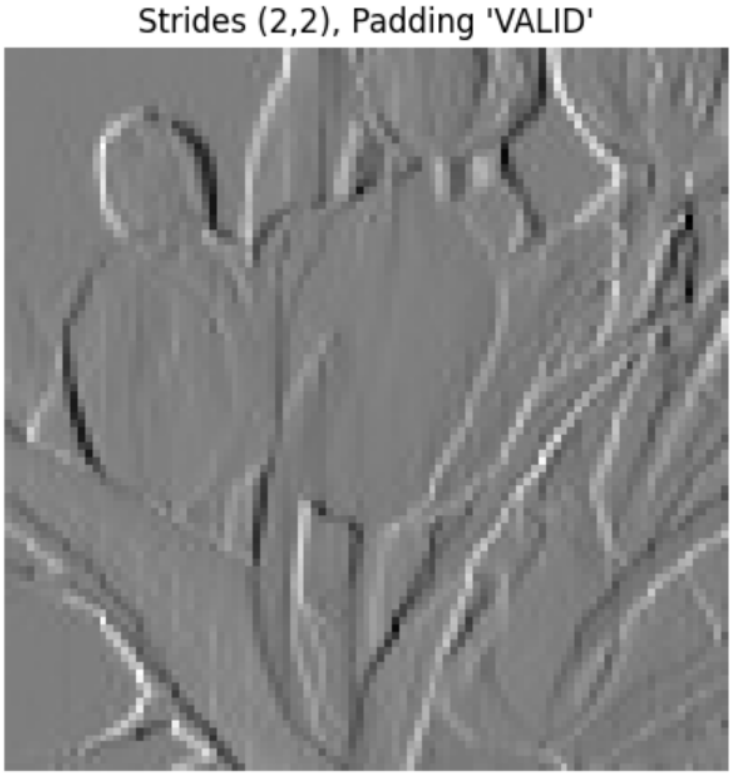
Rysunek 1



Rysunek 2



Rysunek 3



Rysunek 4: Kształt wejściowy obrazu: (200, 200, 1) Kształt wyjściowy po splotu ze strides=(2,2) i padding='VALID': (1, 99, 99, 1)

Stride powoduje zmiane rozdzielczości przetarzanego obrazu natomiast padding usta-

wiony na 'VALID 'nie stosuje żadnego dopełnienia (obraz wyjściowy będzie mniejszy, ponieważ filtr nakłada się tylko tam, gdzie w pełni mieści się na obrazie)

3 zadanie 3

Dane wejściowe:
Wymiar początkowy (krok 0): $24 \times 24 \times 1$ dla `input = np.ones((24, 24, 1))`

Przebieg operacji:

1. **Splot (Conv2D):** 16 filtrów o rozmiarze 3×3, krok $(1, 1)$, padding `SAME`
   - **Kształt wyjściowy:** $24 \times 24 \times 16$
2. **Pooling:** jądro 5×5, krok $(3, 3)$, padding `VALID`
   - **Kształt wyjściowy:** $7 \times 7 \times 16$
   - *Uwaga:* Wymiar przestrzenny wynosi 7×7, a nie 8×8, ponieważ padding `VALID` oznacza brak marginesów, co prowadzi do ucięcia pikseli niemieszczących się w pełnym oknie. Wynika to ze wzoru:
   $$W_{wyj} = \left\lfloor \frac{24 - 5}{3} \right\rfloor + 1 = 6 + 1 = 7$$
3. **Splot (Conv2D):** 32 filtry o rozmiarze 3×3, krok $(1, 1)$, padding `SAME`
   - **Kształt wyjściowy:** $7 \times 7 \times 32$
4. **Pooling:** jądro 3×3, krok $(2, 2)$, padding `VALID`
   - **Kształt wyjściowy:** $3 \times 3 \times 32$
   - *Obliczenie:*
   $$W_{wyj} = \left\lfloor \frac{7 - 3}{2} \right\rfloor + 1 = 2 + 1 = 3$$

Ostateczny wymiar macierzy:

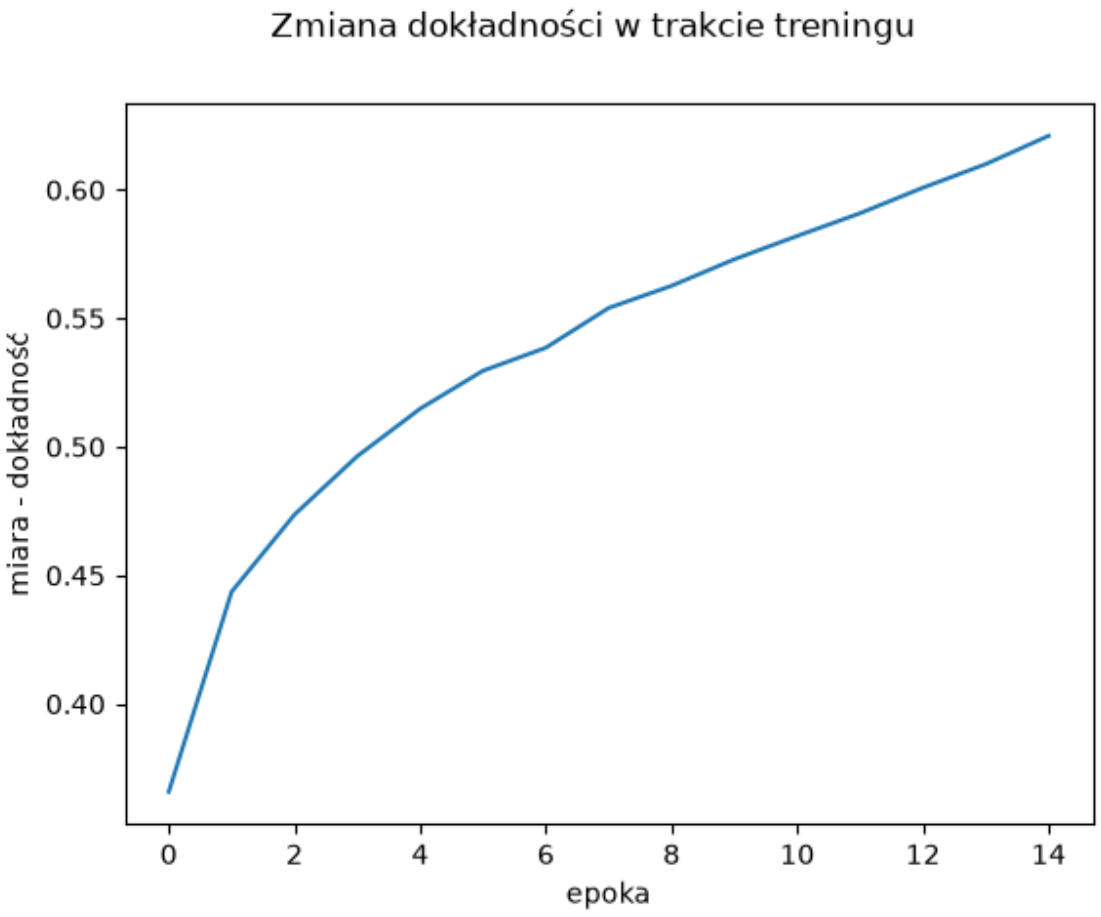
$$3 \times 3 \times 32$$

4 zadanie 4

4.1 zadanie 4a

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 2)	98
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 50)	102,450
dense_1 (Dense)	(None, 10)	510

Rysunek 5



Rysunek 6: Wykres przedstawia proces treningu.

Wynik ewaluacji - dokładność (accuracy): 0.48170000314712524

4.2 zadanie 4b

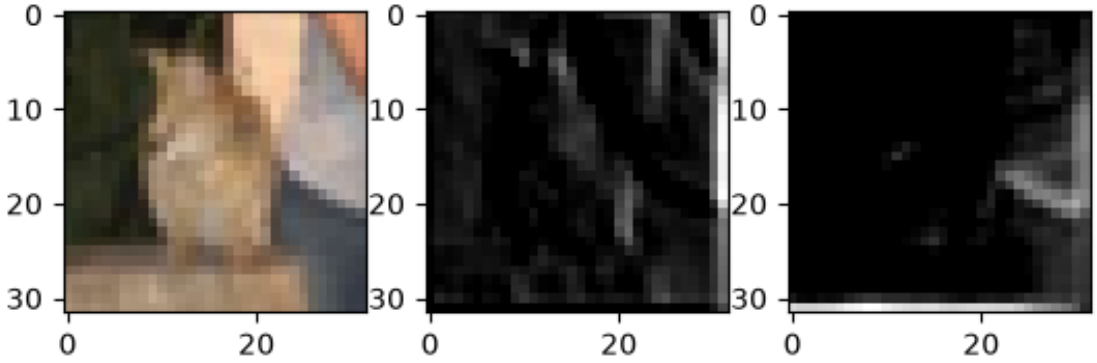
Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 32, 32, 8)	392
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 8)	0
conv2d_4 (Conv2D)	(None, 16, 16, 32)	2,336
max_pooling2d_3 (MaxPooling2D)	(None, 8, 8, 32)	0
flatten_2 (Flatten)	(None, 2048)	0
dense_4 (Dense)	(None, 50)	102,450
dense_5 (Dense)	(None, 10)	510

Rysunek 7



Rysunek 8: Wykres przedstawia proces treningu.

Wynik ewaluacji - dokładność (accuracy): 0.6442000269889832

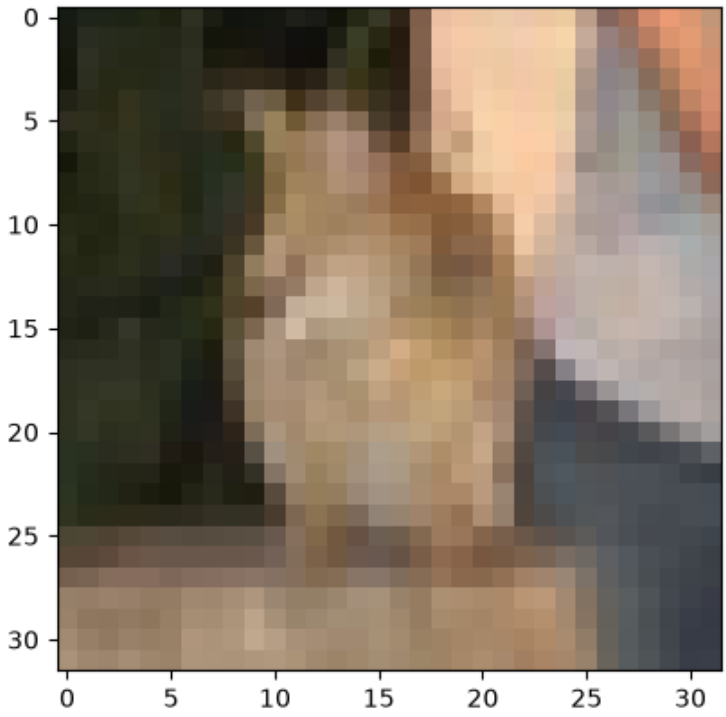


Rysunek 9

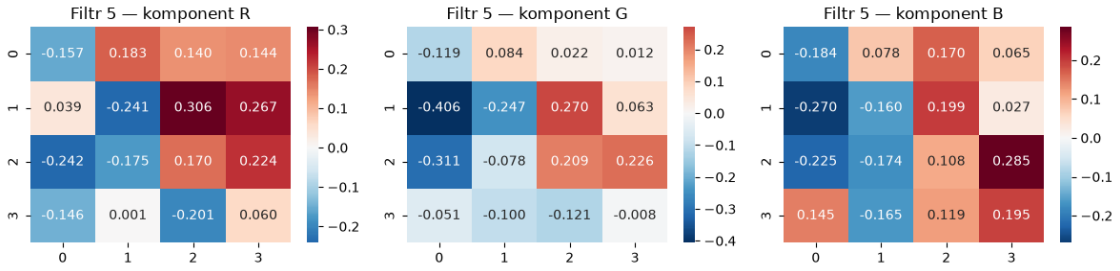
Komentarz do wyników modelu z zadania 4a (prosty model) i zadania 4b - model rozbudowany o 1 dodatkową warstwę splotową i 2 dodatkowe max pooling 2d:

Model 4a uzyskał dokładność 48.2%. Model 4b poprawił ten wynik o około 16% osiągając dokładność 64.4%, co jest zauważalną poprawą, ale wciąż nie jest zadowalającym wynikiem.

4.3 zadanie 4c



Rysunek 10: Przykładowe zdjęcie



Rysunek 11: Mapa ciepła dla 3 kanałów - RGB - Red, Green, Blue

4.4 zadanie 4d

Layer (type)	Output Shape	Param #
conv2d_8 (Conv2D)	(None, 32, 32, 32)	1,568
max_pooling2d_7 (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_9 (Conv2D)	(None, 16, 16, 128)	36,992
max_pooling2d_8 (MaxPooling2D)	(None, 8, 8, 128)	0
conv2d_10 (Conv2D)	(None, 8, 8, 256)	295,168
max_pooling2d_9 (MaxPooling2D)	(None, 4, 4, 256)	0
flatten_4 (Flatten)	(None, 4096)	0
dense_8 (Dense)	(None, 50)	204,850
dense_9 (Dense)	(None, 10)	510

Rysunek 12



Rysunek 13: Wykres przedstawia proces treningu.

Wynik ewaluacji - dokładność (accuracy): 0.7318000197410583
Model z zadania 4d został rozbudowany o trzecią warstwę splotową z 256 filtrami. Po tej trzeciej warstwie konwolucyjnej dodano jeszcze warstwę max pooling 2d.

Model trenował się zauważalnie dłużej od poprzednich.

W modelu z zadania 4d zdecydowanie widać overfitting. Dokładność na zbiorze treningowym wynosi ponad 95%, natomiast na zbiorze testowym wynosi 73.1%. Natomiast ten wynik 73.1% wciąż jest lepszy od modeli 4a czy 4b.